

Riffly — Claude Code Prompts (copy-paste, one per session)

How to use this file:

1. Run **Prompt 0** first in a new empty folder (e.g. `riffly/`). It creates the project and a `CLAUDE.md` memory file so later sessions understand the project instantly.
 2. Then run **Prompt 1**, **Prompt 2**, **Prompt 3** in order — ideally each in a fresh Claude Code session, and only after you've opened and tested the previous phase in a browser.
 3. After each phase, commit to git and publish (Prompt 0 sets up GitHub Pages deployment), so you always have a working version online.
-

PROMPT 0 — Project setup and CLAUDE.md (run first, in an empty folder)

You are setting up a new project called Riffly – a free, zero-cost Progressive Web App (PWA) that teaches a complete beginner (Ishita) to play acoustic guitar. It will later be shared with her friends via a public link, so it must work for multiple independent users at zero cost with no backend server. All user data is stored locally on each user's device (IndexedDB). Target platforms, all first-class: iPhone Safari, Android Chrome, desktop Chrome/Edge.

Do the following now:

1. Initialise a git repository and a vanilla JavaScript project (NO framework – plain HTML/CSS/JS with ES modules; use Vite as the dev server and build tool only). No paid services, no npm packages that require a server. Allowed runtime libraries: pitchy (pitch detection, added in Phase 3), soundtouchjs (time-stretch, added in Phase 3). Nothing else without asking me.
2. Create the folder structure:
 - `index.html`, `/src` (js modules), `/src/styles`, `/assets` (icons, images), `/data` (lessons and chords as JSON), `/public` (manifest, service worker)
3. Create `CLAUDE.md` at the project root containing this project memory (write it verbatim, then add anything else useful):
 - Product: Riffly, a beginner guitar coach PWA. Tagline "Learn. Play. Riff."
 - Design language: warm, encouraging, beginner-friendly. Warm coral/peach primary accent (#E8705A range), soft purple secondary (#7B6BA8 range), generous rounded corners, large touch targets (min 44px), plain non-technical language in all UI

copy. Never intimidating. Mobile-first layout that also works on desktop.

- Architecture rules that must never be violated:

- a. Zero cost: no backend, no database server, no paid API. All personal data in IndexedDB on-device.

- b. One shared timing engine drives ALL rhythmic features (metronome, chord-change trainer, practice studio scrolling). It uses Web Audio API lookahead scheduling (a scheduler that looks ahead ~100ms and schedules sounds on `audioContext.currentTime`). Never use `setInterval`/`setTimeout` for musical timing.

- c. iOS is first-class: audio only starts after a user tap (single global "Tap to enable sound" unlock that resumes the `AudioContext` once per session); install is via Safari Share → Add to Home Screen (show a one-time instruction banner when running in iOS Safari and not installed); request `navigator.storage.persist()` on first run.

- d. Service worker: cache-first for the app shell and data; show a "New version ready – tap to refresh" banner when an update is waiting; display the app version number on the Settings screen.

- e. Left-handed mode: a global setting that mirrors all chord diagrams.

- f. All lesson and chord content lives in `/data` as JSON, never hard-coded in components.

- Phase plan: Phase 1 = shell, timing engine, metronome, chord library, lessons stage 1-2, progress. Phase 2 = chord-change trainer, rhythm generator, song sheets + practice studio, backup/restore, lessons stage 3-4. Phase 3 = mic tuner (pitchy/YIN), audio upload + looping, time-stretch (soundtouchjs), tap-along timing, AI tutor (bring-your-own-key), lessons stage 5.

- Testing rule: after implementing, always run the dev server and check for console errors before declaring done.

4. Create the PWA basics: `manifest.json` (name Riffly, standalone display, theme colour coral, placeholder icons you generate as simple SVG-based PNGs – a coral rounded square with a white "R"), and an empty service worker file registered from `index.html` (real caching logic comes in Phase 1).

5. Set up free deployment: add a GitHub Actions workflow that builds with Vite and deploys the `/dist` folder to GitHub Pages on every push to main. Add a README with two-line deploy instructions.

6. Make an initial commit.

Do not build any features yet. When finished, show me the folder tree and tell me exactly what I need to do on `github.com` to turn on Pages (settings only – do not perform account actions for me).

PROMPT 1 — Phase 1: the reliable core

Read CLAUDE.md first and follow every architecture rule in it. This is Phase 1 of Riffly. Goal: an installable, offline-capable app that already teaches – no microphone features, no AI, no song playback yet.

Build the following, in this order:

1. APP SHELL & NAVIGATION

- Bottom tab navigation (mobile) / side nav (desktop) with five sections: Home, Lessons, Chords, Practice, Progress. Practice shows a friendly "coming in the next update" placeholder for now.
- Home screen: greets the user by first name (ask once via a friendly dialog, store in IndexedDB), shows today's suggested next lesson and current streak.
- Settings screen (gear icon): left-handed mode toggle, sound test button, app version number, and a placeholder section titled "AI Tutor" saying "coming soon".
- First-run states everywhere: no screen may ever be blank. Progress shows "Your journey starts today!" when empty.

2. SERVICE WORKER (real one)

- Cache-first for all app shell files and /data JSON. Version-stamped cache name.
- Update flow: when a new SW is waiting, show a banner "A new version is ready – tap to refresh"; tapping it activates the new worker and reloads.
- Call navigator.storage.persist() on first run and log the result.

3. iOS SUPPORT

- Detect iOS Safari running non-installed (not display-mode: standalone): show a dismissible one-time banner explaining Share → "Add to Home Screen" with a small arrow graphic pointing toward the share button position.
- Global audio unlock: one AudioContext for the whole app, created suspended; the first time any sound feature is used, show a "Tap to enable sound" button that resumes it. After that, never ask again in the session. All audio in the app MUST go through this single context.

4. SHARED TIMING ENGINE (build before metronome – this is the heart)

- A module timingEngine.js implementing Web Audio lookahead scheduling: a scheduler loop (setInterval ~25ms is fine for the LOOP ONLY) that schedules audio events up to ~100ms ahead using audioContext.currentTime.
- API: setTempo(bpm), setTimeSignature(beatsPerBar), start(), stop(), onBeat(callback receiving {bar, beat, audioTime}) for UI sync, and scheduleClick(accented) internally.
- UI callbacks must be driven from the scheduled audio times (drawn via requestAnimationFrame against currentTime), not from the scheduler loop, so visuals

and sound never drift apart.

5. METRONOME (first consumer of the engine)

- Tempo 40-208 BPM via slider + tap-tempo button; time signatures 4/4, 3/4, 2/4, 6/8; accented first beat; large animated beat indicator; volume control.
- Click sounds synthesised with Web Audio (short noise burst / high blip for accent) – no audio files.

6. CHORD LIBRARY

- /data/chords.json with at least these chords: Em, Am, D, G, C, E, A, Dm, F(mini), F(barre), plus B7 and Cadd9 as extras. Each entry: name, difficulty tag, frets per string (x/0/1-4), fingers per string, and 2-3 common-mistake tips.
- Render chord diagrams as SVG generated from the JSON (never static images): 6 strings, 4-5 frets, finger dots with numbers, X/O markers above the nut. Respect the left-handed setting by mirroring.
- Tapping a chord plays it: synthesise the strum with Web Audio – Karplus-Strong plucked-string synthesis per string, staggered ~30ms apart for a natural strum. Route through the shared AudioContext.
- Search box and difficulty filter.

7. LESSONS – STAGES 1 & 2 (content included, written by you now)

- /data/lessons.json. Write the actual lesson text – warm, plain language, short paragraphs.
- Stage 1 (4 lessons): parts of the guitar; how to sit/hold guitar and pick; string names E-A-D-G-B-E with a memory phrase; how to read a chord diagram. Lesson 4 ("Tuning up") explains tuning and says the built-in tuner arrives in a future update – recommend using any free tuner app meanwhile.
- Stage 2 (5 lessons): Em, Am, D, G, C – one lesson each, embedding the live chord diagram + play button from the library, common mistakes, and a "practice this for a few minutes" step. Include, prominently in the Em lesson, the finger-pain talk: fingertips hurting for 1-2 weeks is normal, calluses are forming, 15-20 minutes daily beats one long session.
- Lesson screens: progress dots, Mark Complete button (writes to IndexedDB), next/previous navigation. Locked/unlocked flow is gentle: everything is browsable, but the path highlights the next suggested lesson.

8. PROGRESS TRACKING

- IndexedDB (use a thin promise wrapper you write, ~50 lines – no heavy libraries): store profile, completed lessons, practice days set (for streaks), and settings.
- Progress screen: current streak with a friendly flame/star visual, lessons completed count and list, total practice days.
- Streak logic: a day counts if any lesson was opened or the metronome ran ≥2 minutes that day.

9. QUALITY BAR

- Zero console errors. Lighthouse PWA installability passes. Test the update banner by bumping the SW version. Verify left-handed mirroring on every chord. Keyboard accessible on desktop; 44px minimum touch targets on mobile.

When done: run the dev server, self-review against every numbered item above, fix gaps, then give me a short manual test checklist I can walk through on my phone.

PROMPT 2 — Phase 2: the trainer and the practice studio

Read CLAUDE.md first and follow every architecture rule in it. This is Phase 2 of Riffly. Phase 1 (shell, service worker, iOS support, shared timing engine, metronome, chord library, lessons stages 1-2, progress) is complete – reuse it, never duplicate it. Everything rhythmic below MUST run on the existing shared timing engine.

Build the following, in this order:

1. CHORD-CHANGE TRAINER (inside the Practice tab)

- User picks two chords (defaults suggest the curriculum pairs: Em↔Am, Am↔C, Am↔D, G↔C, G↔D). The trainer shows both diagrams side by side, highlights the current one, and switches the highlight on the bar line, driven by timing-engine beat callbacks.
- Modes: steady tempo, or auto-advance (+4 BPM every 4 bars, ceiling set by user). One-minute "change challenge": count how many clean bars the user claims (self-scored tap) and store a per-pair best score in IndexedDB.
- Encouraging copy throughout; a session here counts toward the practice streak.

2. BACKING-RHYTHM GENERATOR

- Extends the metronome: pattern presets (downstrums-on-beat, D-DU-UDU, folk 3/4, simple rock beat) rendered with synthesised sounds (kick = low sine thump, snare = noise burst, hats = short filtered noise, strum = the Karplus-Strong chord synth playing a user-chosen chord). All scheduled by the timing engine.
- This generator becomes the "app rhythm" backing option for the Practice Studio below – build it as a module with start/stop/setPattern/setChord so the studio can drive it.

3. SONG SHEET FORMAT + PARSER + EDITOR

- Define songSheet JSON: {title, artist?, bpm, timeSignature, key?, lines: [{lyrics?, chords:[{chord, position}], tab?, timeSec?}]} position = character index

in the lyric line where the chord lands. timeSec is optional (used in Phase 3 tap-along; for now line timing is computed from bpm and a linesPerBar setting with per-line overrides).

- ChordPro import: paste text like "[G]Happy birthday to [D]you" → parse to songSheet. Also import the common two-line format (chords on the line above lyrics, aligned by column). Show a live preview during import; unknown chords are flagged, not fatal.

- Built-in editor: edit title/bpm/lines, insert chords via a picker connected to the chord library, live preview. Songs saved to IndexedDB with created/modified dates.

- Ship 3 public-domain starter songs as JSON in /data (write them yourself – e.g. "Happy Birthday", "Amazing Grace", "Oh My Darling Clementine") using only Phase-1 curriculum chords where possible.

4. PRACTICE STUDIO (the heart of the app)

- Song list screen (starter songs + user songs + Import + New buttons), then a player screen.

- Three view modes the user toggles: PLAY (chords + optional tab, large), SING (lyrics karaoke-style), BOTH (chords stacked above lyrics). Current line enlarged and highlighted, previous lines dimmed, auto-scroll keeps the current line in the upper third.

- Backing selector, independent of view mode: App Rhythm (the generator, following the song's chord for each line) or Silent (metronome click only) – "My Audio" appears as a disabled option labelled "next update".

- Transport: play/pause, restart, tempo override (practice slower than written), count-in of one bar before start. Scroll position must be computed from the timing engine's audioTime – never from elapsed wall-clock – so pause/resume stays perfectly in sync.

- Completing a session marks the song "practised" (feeds progress + streak).

5. BACKUP & RESTORE

- Settings → "Back up my data": downloads riffly-backup-YYYY-MM-DD.json containing profile, settings, progress, streaks, scores, and user song sheets (NOT uploaded audio). "Restore": file picker, validate, preview summary ("3 songs, 12 lessons complete – replace current data?"), then import.

6. LESSONS – STAGES 3 & 4 (write the real content now)

- Stage 3 (5 lessons): chord changes intro (links into the trainer with preset pairs); strumming notation (D = down, U = up, - = miss) and first pattern all-downstrums; the D-DU-UDU pattern taught slowly; counting & what 4/4 means; putting pattern + changes together.

- Stage 4 (4 lessons): E, A, Dm one combined pair of lessons; then THE F CHORD lesson – teach mini-F first, state plainly that full barre F takes weeks for everyone and struggling is universal, full barre presented as an optional challenge

at the end.

- Same lesson screen mechanics as Phase 1; embed live trainer/rhythm widgets where a lesson references them.

7. QUALITY BAR

- Studio scrolling stays in sync after pause/resume and tempo change. ChordPro import round-trips the starter songs. Backup→wipe→restore restores everything. Zero console errors; SW version bumped so the update banner appears for existing installs.

When done: self-review against every numbered item, fix gaps, then give me a manual phone test checklist focused on the studio sync and the import flow.

PROMPT 3 — Phase 3: the hard finishers

Read CLAUDE.md first and follow every architecture rule in it. This is Phase 3 (final) of Riffly. Phases 1-2 are complete. This phase adds the four technically hard features plus the AI tutor and final lessons. Take these one at a time, in the order below, and verify each works before moving on.

1. MICROPHONE TUNER

- Add the pitchy library (npm). Build the tuner as a new top-level screen reachable from Home and from Lesson 4.
- getUserMedia with a plain-language permission pre-prompt ("Riffly listens to your string to tell you if it's in tune – nothing is recorded or sent anywhere"). If permission is denied, show exact re-enable instructions per platform (iOS: Settings → Safari/Riffly → Microphone; desktop: the lock icon) – never fail silently.
- Pitch detection via pitchy's autocorrelation (McLeod), analysing ~2048-4096 sample windows from an AnalyserNode on the shared AudioContext. Reject readings with low clarity (<0.9) and outside 60-500 Hz.
- Target standard tuning E2 82.41, A2 110.00, D3 146.83, G3 196.00, B3 246.94, E4 329.63. Auto-detect nearest string; show string name, a needle in cents (-50..+50), flat/sharp hint, and a green "In tune!" state only after $|\text{cents}| \leq 5$ holds for ~1 second.
- Smooth the needle with a rolling median of the last 5 valid readings. Show a "Can't hear you clearly – try somewhere quieter" state when clarity stays low. Add a fallback reference-tone mode: tap a string name to hear the target pitch (synthesised) and tune by ear.
- Update Lesson 4 to use the real tuner.

2. AUDIO UPLOAD + LOOPING (Practice Studio)

- "My Audio" backing option comes alive: file picker for audio files, decoded via `decodeAudioData`, `ArrayBuffer` stored in `IndexedDB` attached to the song. Show per-file size and a total storage meter with a "manage uploads" list (play/delete). Warn kindly above ~200MB total.

- Playback through the shared `AudioContext`. Loop tool: set A/B markers on a waveform strip (render the waveform to a canvas from the decoded buffer), loop the region seamlessly using an `AudioBufferSourceNode` with `loopStart/loopEnd`.

- CRITICAL SYNC RULE: when My Audio backs a song, the studio's scroll position derives from the audio playback position (track seconds), not from the timing engine's bar clock.

3. TAP-ALONG TIMING TOOL

- In the song editor, with My Audio attached: play the track; the user taps once at the start of each line; store `timeSec` per line. Include re-record, nudge $\pm 0.25s$ per line, and clear. Studio uses `timeSec` when present, computed timing otherwise.

4. TIME-STRETCHED SLOW-DOWN

- Add `soundtouchjs`. Speed control 50–100% in 5% steps that preserves pitch, applied to My Audio playback including inside loops. If processing can't keep up on a weak device, fall back automatically to plain `playbackRate` slow-down and label it honestly ("slowed – pitch lowered on this device"). Keep the waveform/loop/scroll sync correct at all speeds (scroll follows the track-time position, which advances slower).

5. AI TUTOR (bring-your-own-key design – no backend, no shared key)

- Settings → AI Tutor: a one-time setup screen with tabbed step-by-step instructions for getting a free API key from (a) Google AI Studio (Gemini free tier) and (b) Groq. The key is stored in `IndexedDB` only and sent only to the provider's own API endpoint directly from the browser. State this privacy fact on the screen.

- Chat UI: available as a tab and as a floating "Ask" button on every lesson screen (pre-filling context like "I'm on the lesson about the G chord"). Streaming responses if the provider supports it; otherwise show a typing indicator.

- System prompt (embed verbatim, then refine): "You are Riffly's guitar tutor. Your student is a complete beginner learning acoustic guitar. Answer in warm, plain, non-technical language. Keep answers short – a few sentences unless asked for more. Be encouraging, never condescending. If a question is not about guitar, music, or practising, gently steer back to guitar."

- Error handling: rate-limit/quota responses show "The tutor has answered a lot today and is resting – it will be back tomorrow. Meanwhile, here are lessons related to your question" with simple keyword-matched lesson links. Invalid key → friendly re-setup link. No key yet → the tutor tab shows the setup screen, and the whole rest of the app works fully without it.

6. LESSONS – STAGE 5 (write the content)

- 3 lessons: "Playing your first real song" (walks through a starter song in the studio); "Bring your own songs" (finding chord sheets online, pasting ChordPro, attaching your own audio, tap-along, loop + slow-down workflow); "Where to go next" (barre chords, fingerpicking, keeping the streak alive) – ending with a congratulations screen tied to the progress tracker.

7. FINAL POLISH & RELEASE

- Full pass on all Phase 1–3 features on a phone-sized viewport; fix layout breaks. Ensure every audio feature routes through the single unlocked `AudioContext` (test on iOS: lock screen mid-playback, return, resume works). Bump the app version and SW cache; confirm the update banner appears over a previous install. Update README with a friendly "share this link with friends" section explaining that each person's data and AI key stay on their own device.

When done: self-review against every numbered item, then give me (a) a manual test checklist ordered by risk (tuner and studio sync first), and (b) a one-paragraph release note I can send to Ishita.

Tips for running these

- Paste one prompt per session; let Claude Code finish and test before the next phase.
- After each phase: `git add -A && git commit -m "Phase N"` and push — GitHub Pages redeploys automatically, so Ishita always has the latest working link.
- If Claude Code asks a clarifying question, answer it rather than repeating the whole prompt.
- If something breaks in a later session, start your message with "Read CLAUDE.md first" — that file carries the project's rules between sessions.